Digital Nurture 5.0 DeepSkilling

Spring Core & Maven

Tasks:

1. **Configuring a Basic Spring Application:**

BookRepository.java

```java
package com.library.repository;


public class BookRepository {

    public BookRepository() {
        System.out.println("BookRepository Bean Created");
    }

    public void displayRepository() {
        System.out.println("Book Repository is Working");
    }

}
```

BookService.java

```java
package com.service.library;


import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void displayService() {

        System.out.println("Book Service is Working");

        bookRepository.displayRepository();
    }

}
```

Application.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.service.library.BookService">

        <property name="bookRepository"
                  ref="bookRepository"/>

    </bean>

</beans>
```

LibraryManagementApplication.java:

```java
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.service.library.BookService;


public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =
                context.getBean("bookService", BookService.class);

        service.displayService();

    }

}
```

Output:

```
BookRepository Bean Created
Book Service is Working
Book Repository is Working
```

2. Implementing Dependency Injection:

BookRepository.java

```java
package com.library.repository;


public class BookRepository {

    public BookRepository() {
        System.out.println("BookRepository Bean Created");
    }

    public void saveBook() {
        System.out.println("Book Repository: Book Saved Successfully");
    }

}
```

BookService.java

```java
package com.service.library;


import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void addBook() {
        System.out.println("Book Service: Processing Book...");
        bookRepository.saveBook();
    }

}
```

applicationContext.java

```xml
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.service.library.BookService">

        <property name="bookRepository"
                  ref="bookRepository"/>

    </bean>

</beans>
```

LibraryManagementApplication.java

```java
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.service.library.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =
                new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean("bookService", BookService.class);

        service.addBook();
    }
}
```

Output:

```
BookRepository Bean Created
Book Service: Processing Book...
Book Repository: Book Saved Successfully
```

---

3. Creating and Configuring a Maven Project

Pom.xml

```xml
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.3.30</version>
</dependency>

<!-- Spring AOP -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>5.3.30</version>
</dependency>

<!-- AspectJ Runtime -->
<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjrt</artifactId>
    <version>1.9.22.1</version>
</dependency>

<!-- AspectJ Weaver -->
<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
    <version>1.9.22.1</version>
</dependency>

<!-- Spring Web MVC -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.3.30</version>
</dependency>
```

Output:

```
[INFO] --- jar:3.4.1:jar (default-jar) @ LibraryManagement ---
[INFO] Building jar: C:\Users\praba\eclipse-workspace\LibraryManagement\target\LibraryManagement-0.0.1-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  6.814 s
[INFO] Finished at: 2026-06-28T19:42:54+05:30
[INFO] ------------------------------------------------------------------------
```

Spring Data JPA and Hibernate:

Tasks:

1. Employee Management System - Overview and Setup

Application.properties

```
spring.application.name=EmployeeManagement
spring.datasource.url=jdbc:h2:mem:testdb

spring.datasource.driverClassName=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=password

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.h2.console.enabled=true
```

EmployeeManagementApplication.java

```
package com.employee;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EmployeeManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeManagementApplication.class, args);
    }

}
```

Output

```
05:30  WARN 33232 --- [EmployeeManagement] [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Theref
05:30  INFO 33232 --- [EmployeeManagement] [           main] o.s.b.h.a.H2ConsoleAutoConfiguration     : H2 console available at '/h2-console'. Database avail
05:30  INFO 33232 --- [EmployeeManagement] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path
05:30  INFO 33232 --- [EmployeeManagement] [           main] c.e.EmployeeManagementApplication        : Started EmployeeManagementApplication in 4.463 second
05:30  INFO 33232 --- [EmployeeManagement] [nio-8080-exec-5] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServ
05:30  INFO 33232 --- [EmployeeManagement] [nio-8080-exec-5] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
```

---

2. **Difference Between Spring Data JPA and Hibernate**

Introduction

Spring Data JPA and Hibernate are both widely used technologies for interacting with relational databases in Java applications. Although they are closely related, they serve different purposes. Hibernate is an Object-Relational Mapping (ORM) framework that implements the Java Persistence API (JPA), whereas Spring Data JPA is a Spring module that simplifies data access by providing repository-based programming.

What is Hibernate?

Hibernate is an open-source ORM framework that maps Java classes to database tables. It simplifies database operations by converting Java objects into database records and vice versa. Hibernate eliminates the need to write most SQL statements manually and provides features such as caching, lazy loading, transaction management, and HQL (Hibernate Query Language).

Features of Hibernate

- **Implements the JPA specification.**
- **Provides Object-Relational Mapping (ORM).**
- **Supports HQL (Hibernate Query Language).**
- **Reduces JDBC boilerplate code.**
- **Provides caching and lazy loading.**
- **Handles transaction management efficiently.**

What is Spring Data JPA?

Spring Data JPA is a Spring framework module that simplifies the implementation of JPA-based repositories. It reduces the amount of boilerplate code required to access databases by providing predefined repository interfaces such as JpaRepository, CrudRepository, and PagingAndSortingRepository.

Features of Spring Data JPA

- **Simplifies JPA implementation.**

- **Provides ready-made CRUD operations.**
- **Supports automatic query generation from method names.**
- **Supports custom queries using @Query.**
- **Supports pagination and sorting.**
- **Integrates seamlessly with the Spring Framework.**

Difference Between Spring Data JPA and Hibernate

Spring Data JPA	Hibernate
A Spring framework module for data access.	An ORM framework.
Built on top of JPA.	Implements the JPA specification.
Uses repository interfaces such as JpaRepository.	Uses SessionFactory and Session.
Automatically generates CRUD methods.	CRUD operations are implemented using Hibernate APIs.
Reduces boilerplate code.	Requires more manual configuration.
Supports pagination and sorting through repositories.	Provides advanced ORM features like caching and lazy loading.
Easier to develop enterprise applications.	Provides detailed control over persistence operations.

Relationship Between Spring Data JPA and Hibernate

Application

↓

Spring Data JPA

↓

JPA Specification

↓

Hibernate

↓

Database

Conclusion

Hibernate is an ORM framework that implements the JPA specification and performs the actual database operations. Spring Data JPA is a higher-level abstraction that simplifies the use of JPA by providing repository interfaces and reducing the amount of code developers need to write. In most Spring Boot applications, Spring Data JPA works together with Hibernate to provide an efficient and easy-to-use persistence layer.

Introduction to JPA (Java Persistence API)

Introduction

Java Persistence API (JPA) is a Java specification used for managing relational data in Java applications. It provides a standard way to map Java objects to database tables using annotations and simplifies database operations without requiring developers to write extensive SQL code.

What is JPA?

JPA (Java Persistence API) is a specification that defines how Java objects are stored, retrieved, updated, and deleted from relational databases. Since JPA is only a specification, it requires an implementation such as Hibernate, EclipseLink, or OpenJPA.

Features of JPA

- **Object-Relational Mapping (ORM).**
- **Standard API for persistence.**
- **Database independence.**
- **Supports CRUD operations.**
- **Supports JPQL (Java Persistence Query Language).**
- **Supports entity relationships.**
- **Supports transaction management.**
- **Integrates well with Spring Boot.**

Important JPA Annotations

@Entity

Marks a Java class as an entity that is mapped to a database table.

@Table

Specifies the name of the database table.

@Id

Represents the primary key of the entity.

@GeneratedValue

Automatically generates values for the primary key.

@Column

Maps a field to a database column.

@OneToMany

Represents a one-to-many relationship between entities.

@ManyToOne

Represents a many-to-one relationship between entities.

@OneToOne

Represents a one-to-one relationship.

@ManyToMany

Represents a many-to-many relationship.

JPA Architecture

Application

↓

JPA API

↓

Hibernate (JPA Provider)

↓

Database

Advantages of JPA

- **Reduces SQL programming.**
- **Improves code maintainability.**
- **Database independent.**
- **Simplifies CRUD operations.**
- **Supports object-oriented programming concepts.**
- **Provides better portability across applications.**
- **Supports advanced querying using JPQL.**

JPA Providers

Some popular JPA providers include:

- **Hibernate**
- **EclipseLink**
- **Apache OpenJPA**

Among these, Hibernate is the most widely used implementation.

Applications of JPA

- **Enterprise Java applications.**
- **Spring Boot applications.**
- **Banking systems.**
- **Employee Management Systems.**
- **E-commerce applications.**
- **Inventory Management Systems.**

Conclusion

Java Persistence API (JPA) provides a standard approach to managing relational data in Java applications. It simplifies database programming by allowing developers to work with Java objects

instead of writing complex SQL statements. Hibernate is the most popular implementation of JPA, and Spring Data JPA further simplifies its usage by providing repository-based programming. Together, they enable developers to build robust, scalable, and maintainable enterprise applications.
